

Zespół Szkół NR.10
Im. Stanisława Staszica

Sprawozdanie z projektu
ThrowMe

Autor: Adam Płochocki

Klasa: 4AP

Warszawa, 10.03.2026r.

Spis treści

1. Cel projektu.....	3
2. Architektura aplikacji.....	4
3. Główne komponenty.....	4
4. Implementacja funkcjonalności.....	6
5. Komunikacja z API (HttpHelper).....	11
6. Wykorzystane biblioteki.....	12

Cel projektu

Celem projektu ThrowMe jest stworzenie aplikacji mobilnej na system Android, która pozwala użytkownikom na mierzenie i porównywanie parametrów rzutu telefonem. Aplikacja wykorzystuje wbudowany akcelerometr urządzenia do detekcji i analizy ruchu w czasie rzeczywistym. Wyniki są zapisywane na serwerze i dostępne przez ogólnodostępny ranking graczy.

Projekt realizuje następujące cele funkcjonalne:

- Rejestrację i logowanie użytkowników z bezpiecznym hashowaniem
- haseł algorytmem SHA-256
- Pomiary rzutów telefonu z wykorzystaniem czujnika akcelerometru
- Obliczanie maksymalnego przyspieszenia, prędkości oraz dystansu
- rzutu
- Wizualizację danych pomiarowych na interaktywnych wykresach liniowych (biblioteka MPAndroidChart)
- Porównanie prędkości rzutu z prędkościami zwierząt
- (AnimalScaleFragment)
- Wyświetlanie globalnej tabeli wyników wszystkich użytkowników
- Przechowywanie i przeglądanie pełnej historii rzutów
- zalogowanego użytkownika

Architektura Aplikacji

Aplikacja została zbudowana w języku Kotlin z wykorzystaniem standardowego stacku Android Jetpack. Architektura opiera się na wzorcu Activity + Fragment, gdzie MainActivity pełni rolę kontenera nawigacyjnego dla trzech głównych fragmentów wyświetlanych przez ViewPager2.

Główne komponenty

Aplikacja składa się z dwóch klas Activity:

1. MainActivity – główny ekran po zalogowaniu. Zarządza paskiem zakładek (Rzut, Ranking, Dane użytkownika), obsługuje nawigację przez ViewPager2 oraz pull-to-refresh przez SwipeRefreshLayout. Udostępnia metodę stopRefreshing() dla fragmentów.
2. LoginActivity – ekran logowania i rejestracji. Waliduje pola wejściowe, hashuje hasło SHA-256 i komunikuje się z API. Po sukcesie zapisuje dane sesji i uruchamia MainActivity.

Fragmenty

1. ThrowFragment – główny ekran pomiaru rzutu. Implementuje

interfejs `SensorEventListener` i rejestruje dane z akcelerometru. Uruchamia algorytm detekcji rzutu oraz po zakończeniu pomiaru wyświetla wyniki i fragment porównania ze zwierzętami.

2. `Leaderboard` – fragment rankingu. Pobiera globalną listę wyników z API i wyświetla ją przez `RecyclerView` z `LeaderboardAdapter`. Obsługuje stany ładowania, błędu i pomyślnego załadowania danych.
3. `UserData` – profil użytkownika. Wyświetla historię rzutów z możliwością filtrowania (po prędkości, przyspieszeniu, dystansie) przez `ChipGroup`. Obsługuje lazy loading wykresów szeregu czasowego oraz przycisk wylogowania.
4. `AnimalScaleFragment` – wizualizacja prędkości rzutu na tle prędkości zwierząt. Skala zawiera: ślimak (0,03 m/s), żółw (0,27 m/s), królik (5 m/s), pies (10 m/s), koń (20 m/s), gepard (29 m/s), sokół (39 m/s). Fragment dobiera dwa najbliższe zwierzęta względem zmierzonej prędkości.

Adaptery `RecyclerView`

3. `LeaderboardAdapter` – wyświetla pozycje rankingowe. Dla miejsc 1–3 stosuje specjalne tło karty i ikonę medalu (  ), dla pozostałych wyświetla numer pozycji tekstem.
4. `ThrowDataAdapter` – wyświetla historię rzutów z rozwijalnym panelem wykresów. Dane szeregu czasowego ładowane są leniwie (lazy loading) dopiero po rozwinięciu elementu listy. Przechowuje indeks aktualnie rozwiniętego elementu (`expandedPosition`).

Implementacja funkcjonalności

System Logowania (LoginActivity.kt)

Ekran logowania/rejestracji obsługuje dwa przepływy – logowanie i rejestrację – przez tę samą metodę `sendRequest(type)`, różniącą się tylko końcówką URL (`/login` lub `/register`).

Przed wysłaniem żądania przeprowadzana jest walidacja pól. Jeśli login lub hasło są puste, rzucony jest błąd przechwytywany przez blok `try/catch` i wyświetlany jako Snackbar przez `PopupNotiHelper`.

Hasło nigdy nie jest wysyłane w formie jawnej. Przed wysłaniem jest hashowane algorytmem SHA-256. Funkcja iteruje po bajtach skrótu i

formatuje każdy bajt jako dwucyfrową wartość szesnastkową (%02x), tworząc 64-znakowy string będący reprezentacją hasha.

Po pozytywnej odpowiedzi serwera (pole uid w JSON) dane sesji są zapisywane w SharedPreferences pod kluczami "username" i "uid", a następnie aplikacja przechodzi do MainActivity metodą startActivity z wywołaniem finish() dla LoginActivity.

Główna nawigacja (MainActivity.kt)

MainActivity implementuje nawigację trzyczakadkową łączącą ViewPager2 z ręcznie rysowanym paskiem przycisków. Aktywna zakładka wyróżniana jest przez pogrubienie tekstu i dynamicznie tworzoną kreseczkę (LayerDrawable) pod etykietą. Pozostałe przyciski wracają do koloru szarego (#8E8E93) i normalnej grubości czcionki.

SwipeRefreshLayout opakowuje całą zawartość – przeciągnięcie w dół wywołuje odpowiednią metodę aktualnie przeglądanej fragmentu: loadLeaderboardData() dla Leaderboard lub loadThrowData() dla

UserData. Klasa udostępnia metodę stopRefreshing() dla fragmentów, by mogły zakończyć animację odświeżania po pobraniu danych.

Synchronizacja między przesuwaniem palcem (swipe) a klikaniem przycisków zakładek jest realizowana przez callback

ViewPager2.OnPageChangeCallback.onPageSelected(), który aktualizuje stan wizualny paska nawigacji.

Pomiar rzutu (ThrowFragment.kt)

Fragment rejestruje się jako SensorEventListener dla czujnika TYPE_ACCELEROMETER z opcją SENSOR_DELAY_FASTEST (najwyższa częstotliwość próbkowania). Algorytm detekcji działa w trzech fazach:

Faza 1 – Oczekiwanie na rzut:

- Aplikacja czeka na przekroczenie progu przyspieszenia $throwThreshold = 20 \text{ m/s}^2$ (ok. 2g). Całkowita wartość przyspieszenia obliczana jest jako norma euklidesowa wektora (x,y,z) pomniejszona o wartość grawitacji ziemskiej:

$$acceleration = \sqrt{x^2 + y^2 + z^2} - GRAVITY_EARTH$$

Faza 2 – Rejestracja danych w locie:

- Po wykryciu rzutu każda próbka czujnika aktualizuje prędkość i pozycję przez całkowanie numeryczne metodą Eulera:


```
velocityX += x * dt  
distanceX += velocityX * dt  
(analogicznie dla osi Y i Z)
```

- Ograniczenie sanitacyjne $dt < 0.1s$ eliminuje fałszywe duże skoki przy przerwach w działaniu procesora. Każda próbka trafia do listy `timeSeriesData` jako obiekt `TimePoint` zawierający: czas od startu rzutu (ms), chwilowe przyspieszenie, prędkość i przebyte dystans.

Faza 3 – Detekcja lądowania:

- Rzut uznawany jest za zakończony, gdy przyspieszenie spada poniżej `landingThreshold = 7 m/s2` i czas trwania przekroczył minimalny próg `minThrowDuration = 200 ms`. Warunek minimalnego czasu eliminuje fałszywe pozytywne detekcje (np. krótkie szarpnięcia).
- Po zakończeniu pomiaru wyniki są wyświetlane użytkownikowi: maksymalne przyspieszenie (m/s²), prędkość (km/h) i dystans (m).
- Poniżej wyników pojawia się `AnimalScaleFragment` z porównaniem.

Użytkownik może:

- Zapisać rzut (Keep) – dane wysyłane są do API jako JSON zawierający `speed`, `accel`, `distance` i pełny szereg czasowy
- Odrzucić rzut (Discard) – UI wraca do stanu początkowego

Fragment Zwierząt (`AnimalScaleFragment.kt`)

Fragment przyjmuje przez Bundle argument `speed` (Float, w m/s). Na podstawie predefiniowanej listy zwierząt posortowanej rosnąco po prędkości wyszukuje dwa najbliższe zwierzęta: jedno o prędkości mniejszej

lub równej zmierzonej i jedno o prędkości większej lub równej. Prędkość wyświetlana jest w km/h po przeliczeniu ($\times 3.6$) i zaokrągleniu do pełnych jednostek.

Tabela wyników (Leaderboard.kt / LeaderboardAdapter.kt)

Fragment pobiera dane przez `HttpHelper.get()` z nagłówkiem token. Wyniki zwracane są jako tablica JSON, iterowana w celu stworzenia listy obiektów `LeaderboardEntry` (position, playerName, score). Adapter stosuje różne style dla top 3:

- Miejsce 1: tło #FFF9E6 (złoty odcień), ikona 🏆, brak numeru
- Miejsce 2: tło #F5F5F5 (srebrny odcień), ikona 🥈, brak numeru
- Miejsce 3: tło #FFF5E6 (brązowy odcień), ikona 🥉, brak numeru
- Pozostałe: białe tło, numer pozycji tekstem

Wszystkie operacje UI po powrocie callbacka sieciowego uruchamiane są na wątku głównym przez `CoroutineScope(Dispatchers.Main).launch{}`.

Dane użytkownika (UserData.kt, ThrowDataAdapter.kt)

Fragment łąduje historię rzutów użytkownika z endpointu `/user/{uid}/data`. Każdy element listy zawiera id, distance, acceleration i speed. Adapter obsługuje rozwijane wiersze – kliknięcie elementu przełącza stan `expandedPosition` i odświeża widok poprzedniego i nowego elementu.

Wykresy ładowane są leniwie: przy rozwinięciu elementu, jeśli `timeSeries == null`, adapter wywołuje callback `onThrowClicked(id)`, który inicjuje

żądanie GET do /user/{uid}/throw/{id}. Po otrzymaniu danych adapter.updateTimeSeries() aktualizuje konkretny element bez przeładowania całej listy.

Dane wyświetlane są na trzech wykresach (MPAndroidChart LineChart):

- Prędkość w czasie (niebieski, #0096FF)
- Przyspieszenie w czasie (pomarańczowy, #FF6400)
- Dystans w czasie (zielony, #00C864)

Każdy wykres obsługuje zoom, przewijanie i szczypanie (pinch zoom). Oś X przedstawia czas w sekundach, oś Y odpowiednią jednostkę.

Filtrowanie historii realizowane jest przez ChipGroup z opcjami:

- Wszystkie – kolejność chronologiczna (domyślna)
- Prędkość – sortowanie malejące po speedVal
- Przyspieszenie – sortowanie malejące po accelerationVal
- Dystans – sortowanie malejące po distanceVal

Komunikacja z API (HttpHelper.kt)

HttpHelper jest singletonem (wzorzec object companion z lazy)

opartym na kliencie OkHttp. Udostępnia dwie metody:

1. get(url, headers?, callback)

Buduje żądanie GET, dodaje opcjonalne nagłówki i wykonuje asynchroniczne wywołanie przez enqueue(). W callbacku zwraca parę (isSuccessful, body).

2. post(url, json, headers?, callback)

Serializuje JSON do RequestBody z typem application/json, buduje żądanie POST i wykonuje analogiczne wywołanie.

Każde żądanie i odpowiedź logowane są do Android Logcat na poziomie DEBUG z tagiem "HTTP", w tym URL, kod odpowiedzi HTTP i treść odpowiedzi. Ułatwia to debugowanie komunikacji podczas rozwoju.

Wykorzystane biblioteki

- androidx.core:core-ktx:1.12.0
Rozszerzenia Kotlin dla API Androida (m.in. SharedPreferences.edit{})
- androidx.appcompat:appcompat:1.6.1
Kompatybilność wsteczna komponentów UI (AppCompatActivity)
- com.google.android.material:material:1.9.0
Komponenty Material Design (Snackbar, ChipGroup, CardView)

- androidx.constraintlayout:constraintlayout:2.1.4
Elastyczne układanie widoków w XML (ConstraintLayout)
- androidx.recyclerview:recyclerview:1.3.2
Wydajna lista przewijalna z recyklingiem widoków (RecyclerView)
- androidx.cardview:cardview:1.0.0
Karty z zaokrąglonymi rogami i cieniem (CardView)
- androidx.swiperefreshlayout:swiperefreshlayout:1.1.0
Gest pull-to-refresh (SwipeRefreshLayout)
- androidx.viewpager2:viewpager2:1.0.0
Przewijany kontener na fragmenty z obsługą gestów (ViewPager2)
- org.jetbrains.kotlin:kotlinx-coroutines-android:1.7.3
Korutyny Kotlin dla asynchronicznych operacji na wątku głównym
- com.squareup.okhttp3:okhttp:4.x
Klient HTTP dla żądań sieciowych (GET, POST)
- com.github.PhilJay:MPAndroidChart:v3.1.0
Biblioteka wykresów (LineChart z obsługą zoom i przewijania)